SA2 - Parte 2 - Engenharia de Software
Gustavo Ferreira da Fonseca - 2669510
Gustavo Henrique Gonçalves - 2669528

1 - PROJETO INICIAL

História de Usuário escolhida para o Projeto Inicial:

1 - Como aluno, eu quero ver os horários disponíveis de monitoria de uma disciplina e reservar um atendimento. (Escolhida para implementação, por isso, fizemos 5 critérios de aceitação).

- **Prioridade**: Alta. Pois é a funcionalidade principal do nosso sistema, permitir o agendamento das monitorias de maneira online.
- **Email**: "A ideia é que o aluno escolha a disciplina, veja os horários livres e reserve um atendimento. “
- **Critérios de Aceitação**:
    1. O sistema deve permitir selecionar uma disciplina disponível.
    2. O sistema deve exibir os horários livres da disciplina selecionada.
    3. O sistema deve permitir que o aluno selecione o horário livre.
    4. O sistema deve reservar o horário selecionado para esse aluno, ou seja, não aparecerá mais em horários disponíveis, pois já está vinculado ao id de um aluno.
    5. O sistema deve exibir para o aluno o horário agendado.

Projeto Inicial de Implementação da História de Usuário Escolhida acima:

1. Descrição em texto: O sistema deve iniciar fazendo o login, quando bem sucedido, terá opções de escolher as disciplinas e mostrar os horários disponíveis com a opção de escolher esse horário para agendar, além de visualizar seus agendamentos, aqui mostrará se continua agendado ou se foi cancelado, além de informações como a matéria e o horário.

2. Estrutura de Classes e Responsabilidades

Classe Atendimento: Ela irá representar os dados e o estado de um atendimento individual.

- **Atributos**: id_atendimento, disciplina, data_hora, status (LIVRE, RESERVADO, CANCELADO) e id_aluno.

- **Métodos**: esta_disponivel(), reservar(id_aluno)

Classe SistemaAgendamento: Gerenciar os atendimentos, exibição das disciplinas, cadastrar horários para atendimentos e etc.

- **Métodos**:

  - cadastrar_horario(atendimento)

  - listar_disciplinas()

  - listar_horarios_livres(disciplina)

  - agendar_horario(id_atendimento, id_aluno)

  - ver_agendamento(id_atendimento)

3. Testes para os critérios de aceitação

Como estamos usando o unittest, criaremos um arquivo teste.py com os seguintes métodos para testar os critérios de aceitação:

Critério de Aceitação 1:

- Teste: test_visualizar_disciplinas_cadastradas

- Objetivo: Quando rodar esse teste, o sistema deve mostrar todas as disciplinas cadastrados no programa de monitoria.

Critério de Aceitação 2:

- Teste: test_exibir_horarios_livres

- Objetivo: Quando eu consultar deve aparecer apenas os horários de agendamentos com o status = “LIVRE”.

Critério de Aceitação 3:

- Teste: test_selecionar_horario_livre

- Objetivo: O aluno deve conseguir selecionar um horário livre de agendamento.

Critério de Aceitação 4:

- Teste Sucesso: test_realizar_agendamento_sucesso
- Teste Falha: test_realizar_agendamento_falha

- Objetivo: Quando o aluno selecionar o horário livre e agendar, deve gerar um agendamento, mudando o status de LIVRE para RESERVADO, vincular os dados do aluno ao atendimento/objeto criado e não listar mais aquele horário como disponível. No caso de falha tentaremos realizar um atendimento em um horário já reservado.

Critério de Aceitação 5:

- Teste: test_mostrar_agendamento

- Objetivo: Ao entrar no menu de meus agendamentos deve aparecer para o aluno seus agendamentos, o horário, a disciplina e o id_agendamento.

2 - EXECUÇÃO

Instruções para execução

O protótipo foi desenvolvido em Python e utiliza apenas recursos da biblioteca padrão e o unittest, necessitando ter o Python 3 instalado.
Os arquivos devem estar na mesma pasta, por exemplo:

- Atendimento.py (classe Atendimento) e SistemaAgendamento.py (classe SistemaAgendamento) e teste.py.

Para verificar se os arquivos estão sintaticamente corretos, abra o terminal na pasta do projeto e execute:

python -m unittest discover -s codigo

Ou também apenas:

python teste.py dentro da pasta código

Comparação entre o planejamento e a implementação

A implementação apresentada segue a estrutura prevista no projeto inicial. A classe Atendimento representa cada horário e mantém os atributos de identificação, aluno, disciplina, data e hora e status. O método esta_disponivel verifica se o atendimento está LIVRE, e reservar(id_aluno) altera o status para RESERVADO e associa o identificador do aluno, impedindo uma nova reserva pelo método reservar quando o horário já não está livre.

A classe SistemaAgendamento mantém os atendimentos em um dicionário, disponibiliza a lista de disciplinas, permite cadastrar horários, listar horários livres por disciplina, realizar agendamentos e exibir os dados de um atendimento. Os dados também são escritos no arquivo DadosAtendimentos. Assim, as principais operações previstas para a história de usuário estão representadas nas classes fornecidas.

Uma das diferenças entre o planejamento e a implementação foi a previsão inicial de uma interface gráfica para a visualização do sistema Porém a ideia foi descartada devido a falta de informação sobre o código não poder possuir nenhum tipo de input ou interface para poder aplicar os testes. Portanto o código teve que ser adaptado para poder não depender de tais ferramentas.

3 - AUTOAVALIAÇÃO

Foi um projeto bem interessante. Aprendemos sobre o unittest, algo que nunca tinhamos usado antes. Também foi importante para melhorarmos nossa percepção de trabalho e ajustar o planejamento. Pois no planejamento inicial pensamos em muitas coisas, até mesmo em desenvolver parte das demais histórias de usuário e quase não deu tempo de finalizar uma história de usuário. Sem dúvidas foi muito útil para melhorarmos para o projeto final da disciplina.

Houveram mudanças também no cronograma, devido a uma viagem que fiz a trabalho na segunda e terça atrasei em um dia a finalização dos testes.